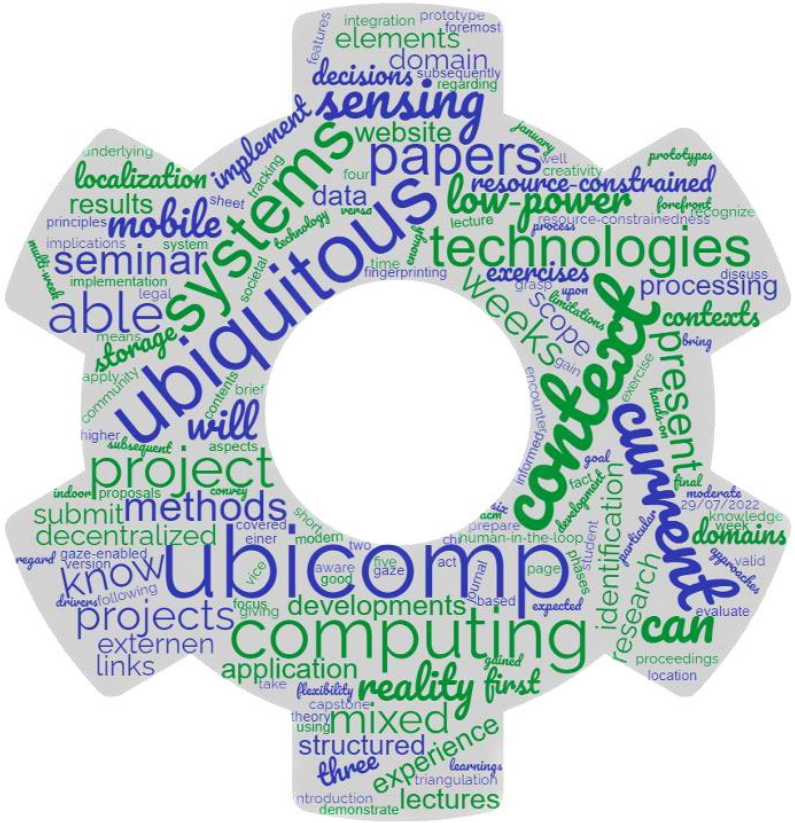
Universität St.Gallen

Distributed Systems, 3,902, Bachelor of Computer Science

Spark execution in the History Server

Jan Grau, Jérémy Lemée, Andrei Ciortea



Universität St.Gallen

Session plan

`Platforms`	RDD partitioning
`Timetable`	lazy evaluation
`Departures`	file-based partitioning
`LateTrains`	narrow transformations
`DelayMinutes`	wide transformations
`SumThreeWays`	map-side aggregation
`StationNames`	RDD joins
`WorstStations`	shuffle reuse
`Punctuality`	RDD caching



SBB Re 460 between Vevey and Chexbres. Photo: Kabelleger / David Gubler, CC BY-SA 4.0, Wikimedia Commons

Universität St.Gallen

Dataset

The generated dataset contains departures at 40 Swiss stations between January and June 2025. Each record includes the delay in minutes.

data/trains/2025-01.csv ... 2025-06.csv date,time,line,station_id,delay_min

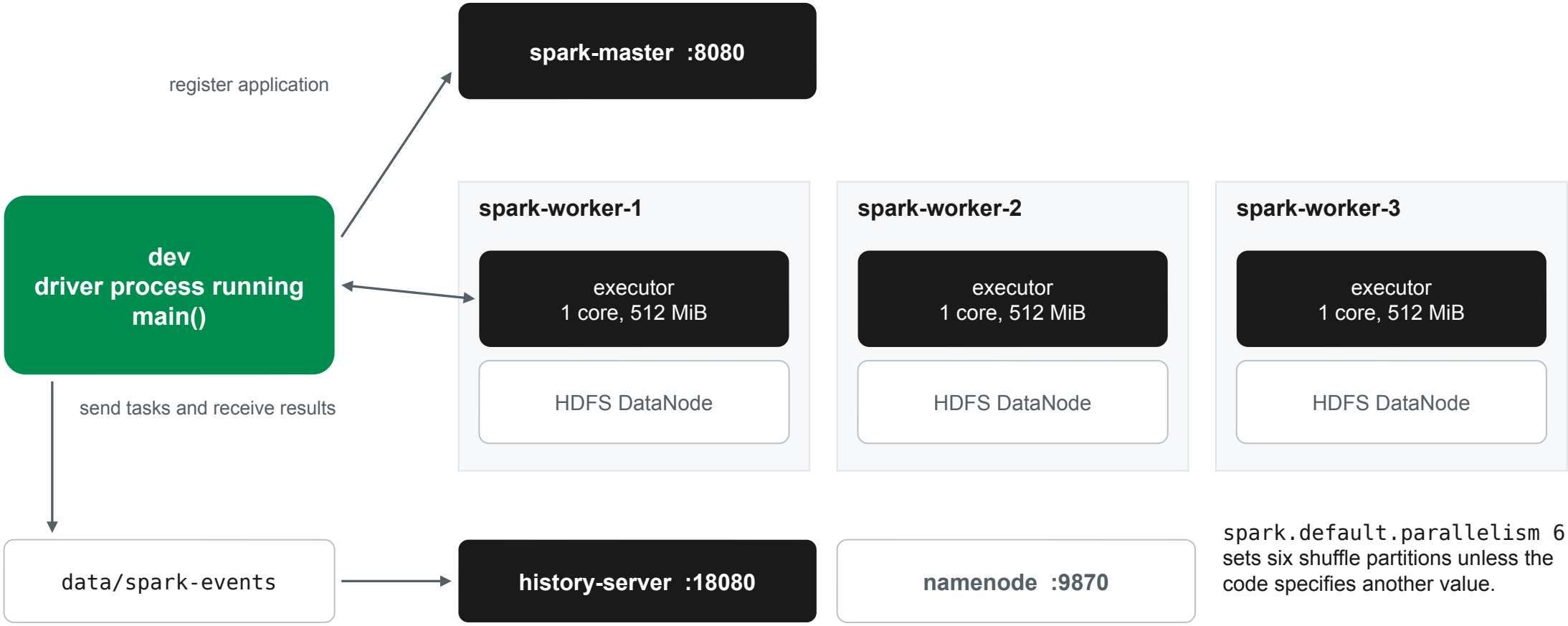
```
2025-01-01,00:00,IR13,19,1
2025-01-01,00:01,IC5,1,0
2025-01-01,00:02,S3,29,3
2025-01-01,00:03,IR15,9,0
2025-01-01,00:04,RE33,16,0
```

data/stations.csv station_id,name

```
1,Zürich HB
2,Bern
3,Basel SBB
```

Universität St.Gallen

Cluster setup



`spark.default.parallelism 6` sets six shuffle partitions unless the code specifies another value.

The driver records events during the run. The History Server loads the event log after the application exits.

Universität St.Gallen

Run the examples

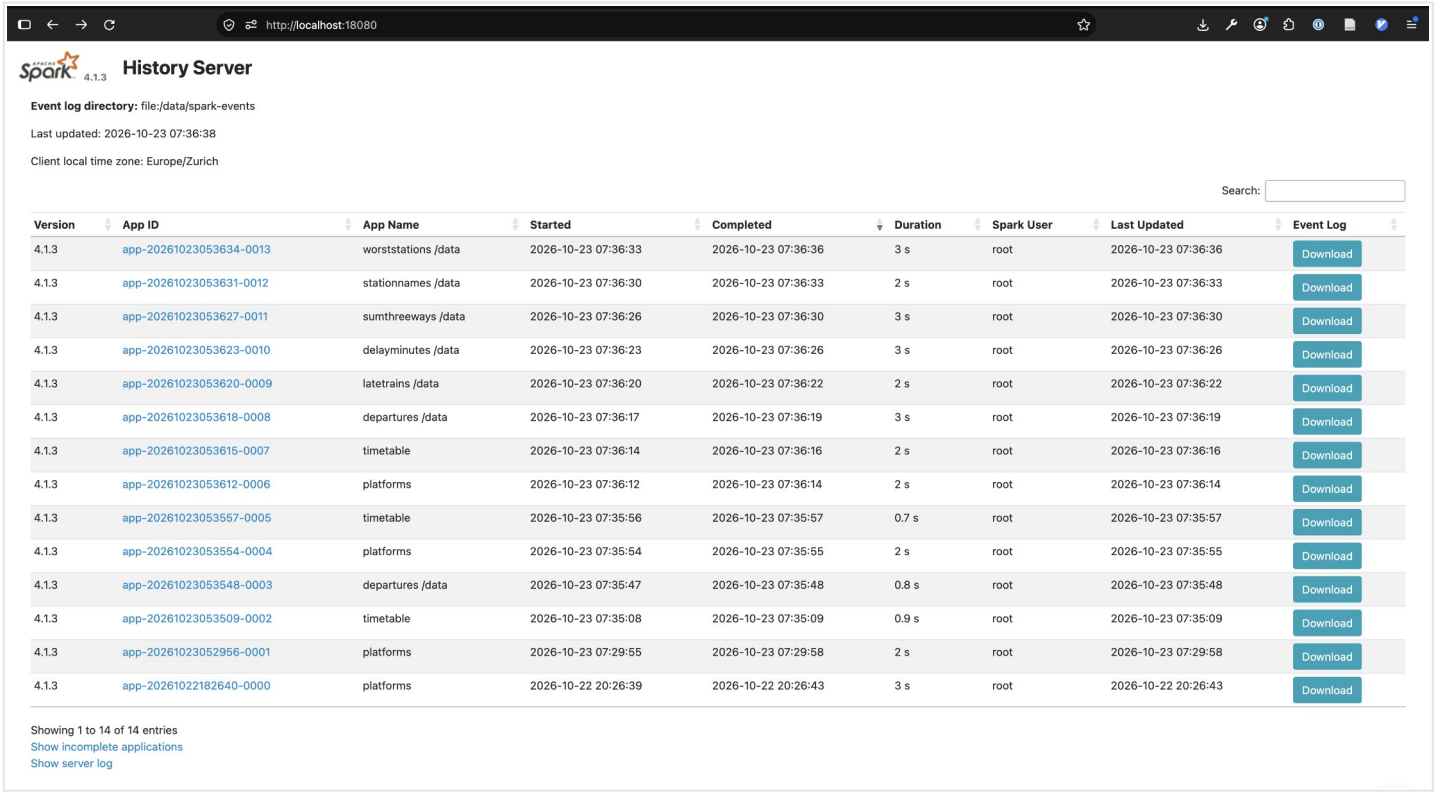
The exercise is a template repository on GitHub: HSG-DS-HS26/BCS-DS-Exercise-Session-6. Select **Use this template** to copy it into your own repository.

```
./run.sh platforms            # exercise/01_platforms/Platforms.java
./run.sh solution platforms   # solution/01_platforms/Platforms.java
./run.sh solution all         # every program in solution/
```

Open each cluster UI at `http://localhost:<port>`. In Codespaces/Devcontainer, use the Ports tab.

Universität St.Gallen

Read the History Server



Navigation

- Each run creates an application named after the program and data path. Select **Show incomplete applications** to include active runs.
- Open the App ID. Use **Jobs** for job timing, **Stages** for task and shuffle metrics, **Storage** for cached RDDs, **Executors** for worker activity, and **DataFlint** for the application summary.

Universität St.Gallen

Spark execution units

application	a program run with a single `SparkContext`
job	work triggered by an action
stage	a set of tasks bounded by shuffle operations
task	the work for a single partition on an executor core

Green boxes show tasks; arrows between stages show shuffles.



Universität St.Gallen

From last week

Resilient distributed datasets

```
nums = sc.parallelize(List.of(1, 2, 3, 4, 5, 6, 7, 8, 9), 3) a JavaRDD<Integer> in 3 partitions
```

1, 2, 3	4, 5, 6	7, 8, 9
partition 0 on spark-worker-1	partition 1 on spark-worker-2	partition 2 on spark-worker-3

Actions in the Java API

operation	example	result
`count()`	`nums.count()`	`4`, as a `long`

Universität St.Gallen

Platforms

Distribute nine platform numbers across three partitions and count them.



```
JavaSparkContext sc = new JavaSparkContext(new SparkConf().setAppName("platforms"));

// TODO start: put the platform numbers 1 to 9 into an RDD with 3 partitions
JavaRDD<Integer> platforms = null;
// TODO end

System.out.println("partitions: " + platforms.getNumPartitions());
// count is an action: it starts a job with a task per partition.
System.out.println("platforms: " + platforms.count());
```

Universität St.Gallen

Platforms

Distribute nine platform numbers across three partitions and count them.



```
JavaSparkContext sc = new JavaSparkContext(new SparkConf().setAppName("platforms"));

// TODO start: put the platform numbers 1 to 9 into an RDD with 3 partitions
JavaRDD<Integer> platforms = sc.parallelize(List.of(1, 2, 3, 4, 5, 6, 7, 8, 9), 3);
// TODO end

System.out.println("partitions: " + platforms.getNumPartitions());
// count is an action: it starts a job with a task per partition.
System.out.println("platforms: " + platforms.count());
```

Universität St.Gallen

Platforms

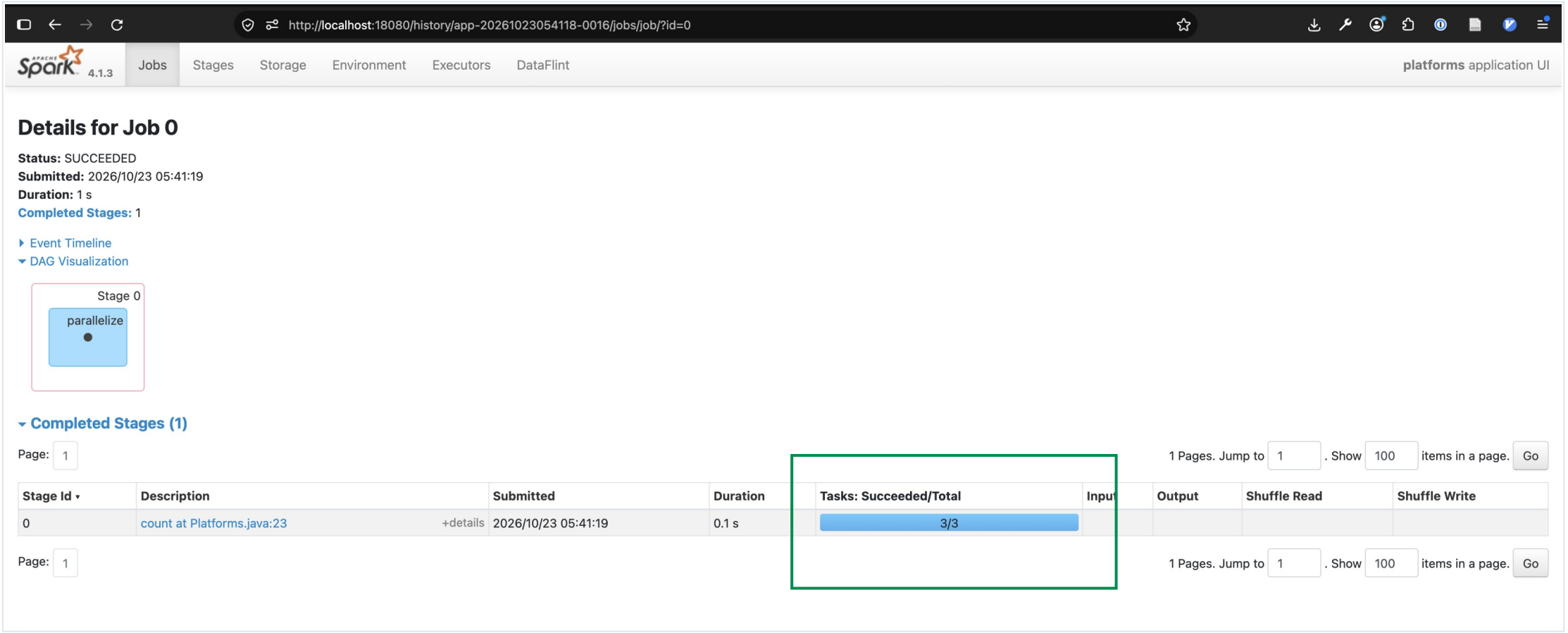
```
JavaRDD<Integer> platforms = sc.parallelize(List.of(1, 2, 3, 4, 5, 6, 7, 8, 9), 3);
System.out.println("partitions: " + platforms.getNumPartitions());
System.out.println("platforms: " + platforms.count());
```

Prediction

- How many jobs and stages will Spark create?
- Which values will each task process?

Universität St.Gallen

Let's see



Universität St.Gallen

Platforms

```
JavaRDD<Integer> platforms = sc.parallelize(List.of(1, 2, 3, 4, 5, 6, 7, 8, 9), 3);
System.out.println("partitions: " + platforms.getNumPartitions());
System.out.println("platforms: " + platforms.count());
```

Prediction

- How many jobs and stages will Spark create?
- Which values will each task process?

Observed result

jobs	1: count
stages	1
tasks	3

`parallelize(..., 3)` creates the partitions 1-3, 4-6, and 7-9. The stage runs a task for each partition.

Universität St.Gallen

From last week

Transformations on JavaRDD

operation	example	result
`map(f)`	`nums.map(x -> x * x)`	1, 4, 9, 16: exactly 1 result per element
`filter(f)`	`nums.filter(x -> x % 2 == 0)`	2, 4: the elements for which `f` returns true

Actions in the Java API

operation	example	result
`collect()`	`nums.collect()`	[5, 3, 8, 1] as a `List` on the driver; for small results only

Universität St.Gallen

Timetable

Filter the even platform numbers and convert them to labels.



```
JavaSparkContext sc = new JavaSparkContext(new SparkConf().setAppName("timetable"));

JavaRDD<Integer> platforms = sc.parallelize(List.of(1, 2, 3, 4, 5, 6, 7, 8, 9), 3);

// TODO start: keep the even platforms and label each as "platform <number>"
JavaRDD<Integer> even = null;
JavaRDD<String> labels = null;
// TODO end
System.out.println("transformations defined, no job has started yet");

System.out.println("count: " + labels.count());
System.out.println("collect: " + labels.collect());
```

Universität St.Gallen

Timetable

Filter the even platform numbers and convert them to labels.



```
JavaSparkContext sc = new JavaSparkContext(new SparkConf().setAppName("timetable"));

JavaRDD<Integer> platforms = sc.parallelize(List.of(1, 2, 3, 4, 5, 6, 7, 8, 9), 3);

// TODO start: keep the even platforms and label each as "platform <number>"
JavaRDD<Integer> even = platforms.filter(p -> p % 2 == 0);
JavaRDD<String> labels = even.map(p -> "platform " + p);
// TODO end
System.out.println("transformations defined, no job has started yet");

System.out.println("count: " + labels.count());
System.out.println("collect: " + labels.collect());
```

Universität St.Gallen

Timetable

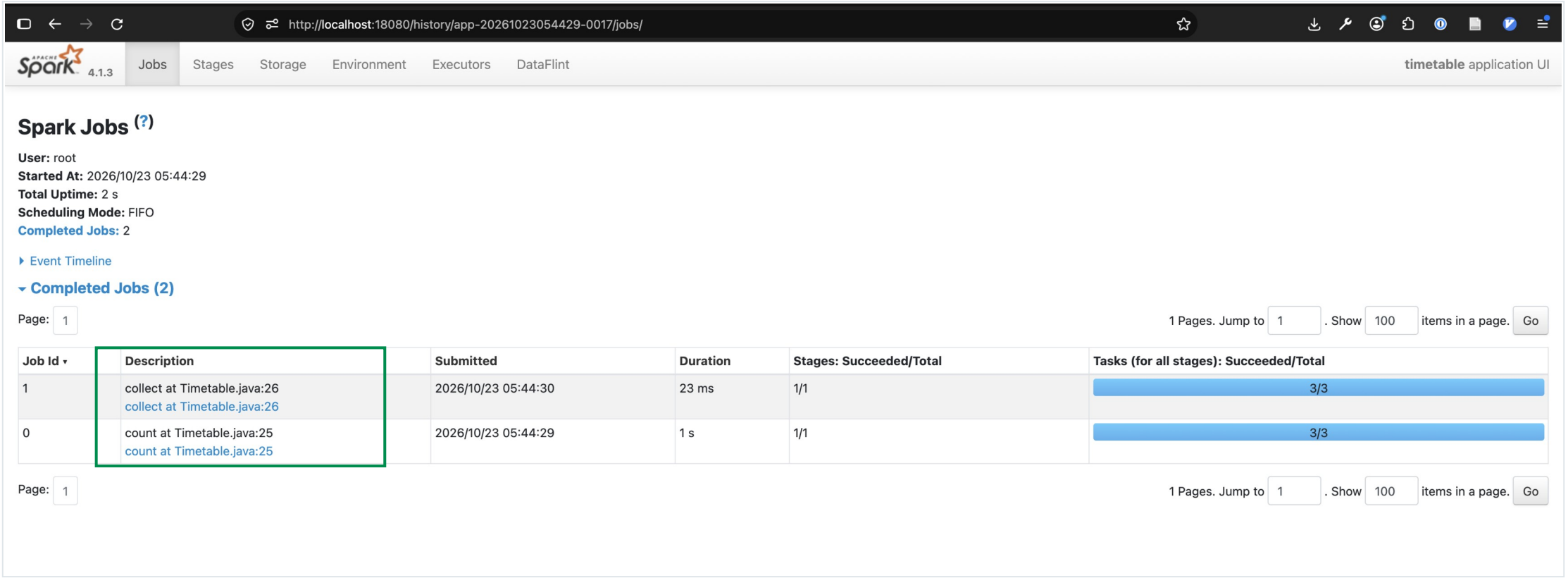
```
JavaRDD<Integer> platforms = sc.parallelize(List.of(1, 2, 3, 4, 5, 6, 7, 8, 9), 3);
JavaRDD<Integer> even = platforms.filter(p -> p % 2 == 0);
JavaRDD<String> labels = even.map(p -> "platform " + p);
System.out.println("transformations defined, no job has started yet");
System.out.println("count: " + labels.count());
System.out.println("collect: " + labels.collect());
```

Prediction

- How many jobs will run?
- Which message appears first: `transformations defined` or `Starting job`?
- What will `collect` return?

Universität St.Gallen

Let's see



Universität St.Gallen

Timetable

```
JavaRDD<Integer> platforms = sc.parallelize(List.of(1, 2, 3, 4, 5, 6, 7, 8, 9), 3);
JavaRDD<Integer> even = platforms.filter(p -> p % 2 == 0);
JavaRDD<String> labels = even.map(p -> "platform " + p);
System.out.println("transformations defined, no job has started yet");
System.out.println("count: " + labels.count());
System.out.println("collect: " + labels.collect());
```

Prediction

- How many jobs will run?
- Which message appears first: `transformations defined` or `Starting job`?
- What will `collect` return?

Observed result

jobs	2: count, collect
stages	1 per job
tasks	3 per stage

`transformations defined` appears first. `count` returns `4`. `collect` returns `[platform 2, platform 4, platform 6, platform 8]`.

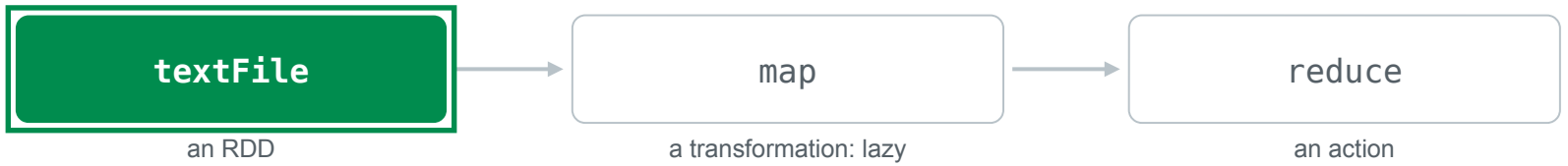
From last week

Transformations and actions

```
JavaSparkContext sc = new JavaSparkContext(new SparkConf().setAppName("line-lengths"));

JavaRDD<String> lines = sc.textFile("data.txt");
JavaRDD<Integer> lineLengths = lines.map(s -> s.length());
int totalLength = lineLengths.reduce((a, b) -> a + b);

System.out.println(totalLength + " characters");
```



Universität St.Gallen

Departures

Read the monthly files and count all departure records.



```
String base = args.length > 0 ? args[0].replaceAll("/+$", "") : "/data";

JavaSparkContext sc = new JavaSparkContext(new SparkConf().setAppName("departures " + base));

// TODO start: read every file in base + "/trains"
JavaRDD<String> lines = null;
// TODO end

// The driver lists the files to find the partitions; no job runs for this.
System.out.println("partitions: " + lines.getNumPartitions());
System.out.println("departures: " + lines.count());
```

Universität St.Gallen

Departures

Read the monthly files and count all departure records.



```
String base = args.length > 0 ? args[0].replaceAll("/+$", "") : "/data";

JavaSparkContext sc = new JavaSparkContext(new SparkConf().setAppName("departures " + base));

// TODO start: read every file in base + "/trains"
JavaRDD<String> lines = sc.textFile(base + "/trains");
// TODO end

// The driver lists the files to find the partitions; no job runs for this.
System.out.println("partitions: " + lines.getNumPartitions());
System.out.println("departures: " + lines.count());
```

Universität St.Gallen

A bit of Theory

`textFile` makes Hadoop make at least 2 splits. Each split is a partition for a single task.

goal	the total input of all files / 2
split size	min(goal, block size): the local file system reports blocks of 32 MiB, HDFS in this cluster stores blocks of 8 MiB
cuts	Hadoop cuts each file on its own into pieces of the split size until the rest is at most 1.1 x the split size, which becomes the last split.

A 100 MiB file with a split size of 32 MiB



The last 36 MiB are more than 1.1 x 32 MiB, so Hadoop cuts them once more.

Universität St.Gallen

Departures

```
JavaRDD<String> lines = sc.textFile(base + "/trains");
System.out.println("partitions: " + lines.getNumPartitions());
System.out.println("departures: " + lines.count());
```

Prediction

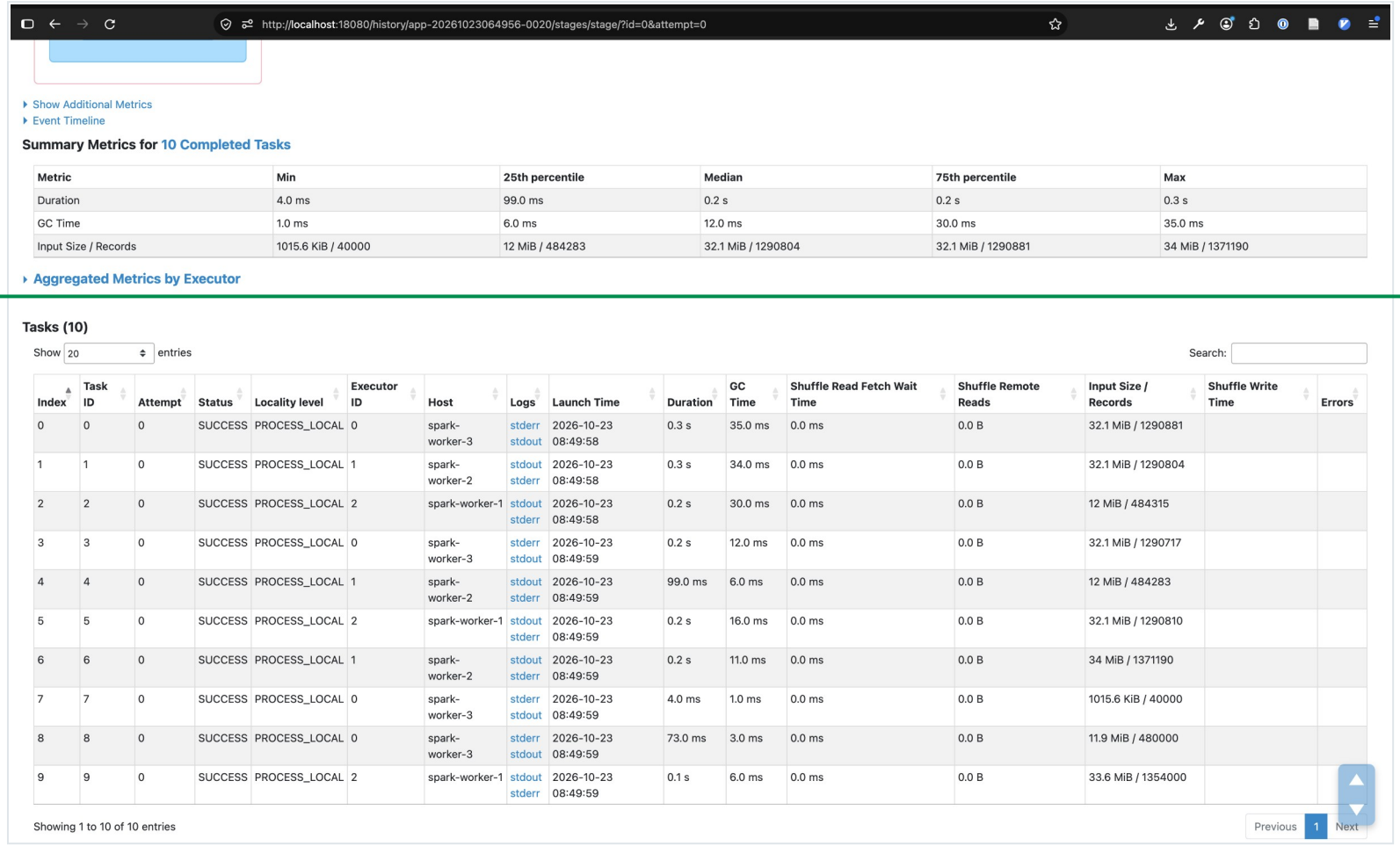
- How many partitions will each file give?
- Will `getNumPartitions` start a job?
- What Input value (in MiB) will the History Server show?

Files

2025-01.csv	1.0 MiB
2025-02.csv	11.9 MiB
2025-03.csv	33.6 MiB
2025-04.csv	44.0 MiB
2025-05.csv	66.0 MiB
2025-06.csv	76.0 MiB
total	232.5 MiB

Universität St.Gallen

Let's see



Metric	Min	25th percentile	Median	75th percentile	Max
Duration	4.0 ms	99.0 ms	0.2 s	0.2 s	0.3 s
GC Time	1.0 ms	6.0 ms	12.0 ms	30.0 ms	35.0 ms
Input Size / Records	1015.6 KiB / 40000	12 MiB / 484283	32.1 MiB / 1290804	32.1 MiB / 1290881	34 MiB / 1371190

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Read Fetch Wait Time	Shuffle Remote Reads	Input Size / Records	Shuffle Write Time	Errors
0	0	0	SUCCESS	PROCESS_LOCAL	0	spark-worker-3	stderr stdout	2026-10-23 08:49:58	0.3 s	35.0 ms	0.0 ms	0.0 B	32.1 MiB / 1290881		
1	1	0	SUCCESS	PROCESS_LOCAL	1	spark-worker-2	stdout stderr	2026-10-23 08:49:58	0.3 s	34.0 ms	0.0 ms	0.0 B	32.1 MiB / 1290804		
2	2	0	SUCCESS	PROCESS_LOCAL	2	spark-worker-1	stdout stderr	2026-10-23 08:49:58	0.2 s	30.0 ms	0.0 ms	0.0 B	12 MiB / 484315		
3	3	0	SUCCESS	PROCESS_LOCAL	0	spark-worker-3	stderr stdout	2026-10-23 08:49:59	0.2 s	12.0 ms	0.0 ms	0.0 B	32.1 MiB / 1290717		
4	4	0	SUCCESS	PROCESS_LOCAL	1	spark-worker-2	stdout stderr	2026-10-23 08:49:59	99.0 ms	6.0 ms	0.0 ms	0.0 B	12 MiB / 484283		
5	5	0	SUCCESS	PROCESS_LOCAL	2	spark-worker-1	stdout stderr	2026-10-23 08:49:59	0.2 s	16.0 ms	0.0 ms	0.0 B	32.1 MiB / 1290810		
6	6	0	SUCCESS	PROCESS_LOCAL	1	spark-worker-2	stdout stderr	2026-10-23 08:49:59	0.2 s	11.0 ms	0.0 ms	0.0 B	34 MiB / 1371190		
7	7	0	SUCCESS	PROCESS_LOCAL	0	spark-worker-3	stderr stdout	2026-10-23 08:49:59	4.0 ms	1.0 ms	0.0 ms	0.0 B	1015.6 KiB / 40000		
8	8	0	SUCCESS	PROCESS_LOCAL	0	spark-worker-3	stderr stdout	2026-10-23 08:49:59	73.0 ms	3.0 ms	0.0 ms	0.0 B	11.9 MiB / 480000		
9	9	0	SUCCESS	PROCESS_LOCAL	2	spark-worker-1	stdout stderr	2026-10-23 08:49:59	0.1 s	6.0 ms	0.0 ms	0.0 B	33.6 MiB / 1354000		

Universität St.Gallen

Departures

```
JavaRDD<String> lines = sc.textFile(base + "/trains");
System.out.println("partitions: " + lines.getNumPartitions());
System.out.println("departures: " + lines.count());
```

Prediction

- How many partitions will each file give?
- Will `getNumPartitions` start a job?
- What Input value (in MiB) will the History Server show?

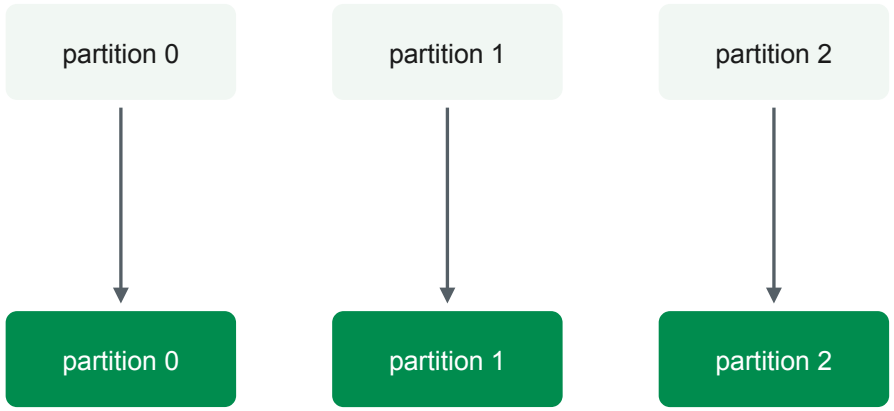
Observed result

2025-01.csv	1.0 MiB	1
2025-02.csv	11.9 MiB	1
2025-03.csv	33.6 MiB	1: at most 1.1 x 32 MiB
2025-04.csv	44.0 MiB	2: 32 + 12.0
2025-05.csv	66.0 MiB	2: 32 + 34.0
2025-06.csv	76.0 MiB	3: 32 + 32 + 12.0
total	232.5 MiB	10 partitions, 10 tasks

Universität St.Gallen

Wide vs Narrow Transformations

Narrow dependency

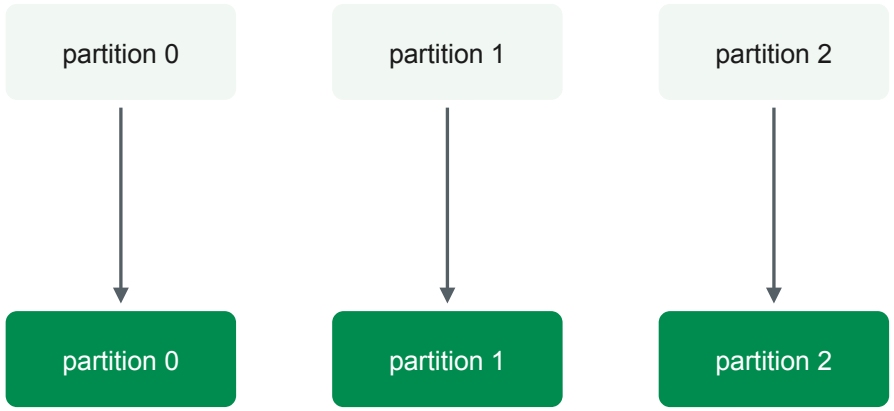


Each output partition reads a single input partition.
`filter`, `map`, `mapToPair`, `mapValues`
Spark keeps these operations in a single stage.

Universität St.Gallen

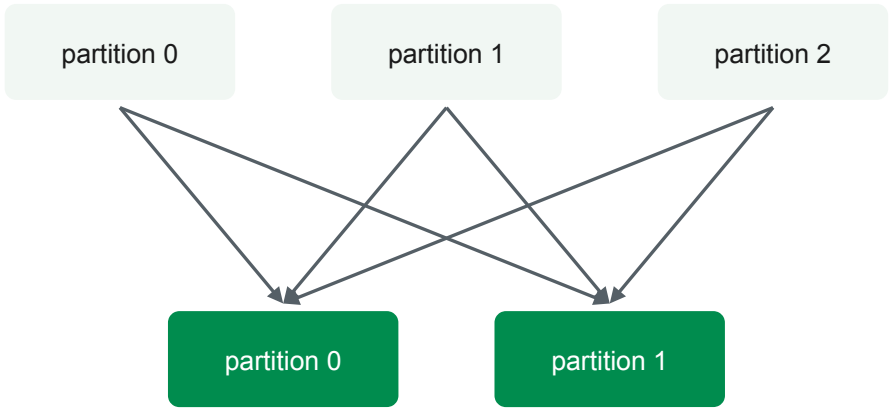
Wide vs Narrow Transformations

Narrow dependency



Each output partition reads a single input partition.
`filter`, `map`, `mapToPair`, `mapValues`
Spark keeps these operations in a single stage.

Wide dependency



Each output partition reads multiple input partitions.
`reduceByKey`, `groupByKey`, `join`, `sortByKey`
A **shuffle** starts the next stage.

Universität St.Gallen

From last week

Transformations on JavaRDD

operation	example	result
`mapToPair(f)`	`nums.mapToPair(x -> new Tuple2<>(x % 2, x))`	(1, 1), (0, 2), (1, 3), (0, 4): the key is `x % 2`

Universität St.Gallen

LateTrains

Filter delayed departures and map each record to `(station, delay)`.



```
JavaRDD<String> lines = sc.textFile(base + "/trains");

// TODO start: keep the late departures and turn each into (station, delay)
JavaPairRDD<Integer, Integer> late = null;
// TODO end

// The lineage: each RDD above the RDD it is computed from.
System.out.println(late.toDebugString());
System.out.println("late departures: " + late.count());
```

data/trains/2025-01.csv

```
2025-01-01,00:00,IR13,19,1
2025-01-01,00:01,IC5,1,0
2025-01-01,00:02,S3,29,3
2025-01-01,00:03,IR15,9,0
2025-01-01,00:04,RE33,16,0
2025-01-01,00:05,IC5,8,1
```

common/Trains.java

```
public static int station(String line) {
    return Integer.parseInt(line.split(",")[3]);
}

public static int delay(String line) {
    return Integer.parseInt(line.split(",")[4]);
}
```

Universität St.Gallen

LateTrains

Filter delayed departures and map each record to `(station, delay)`.



```
JavaRDD<String> lines = sc.textFile(base + "/trains");

// TODO start: keep the late departures and turn each into (station, delay)
JavaPairRDD<Integer, Integer> late = lines
    .filter(line -> Trains.delay(line) > 0)
    .mapToPair(line -> new Tuple2<>(Trains.station(line), Trains.delay(line)));
// TODO end

// The lineage: each RDD above the RDD it is computed from.
System.out.println(late.toDebugString());
System.out.println("late departures: " + late.count());
```

Universität St.Gallen

LateTrains

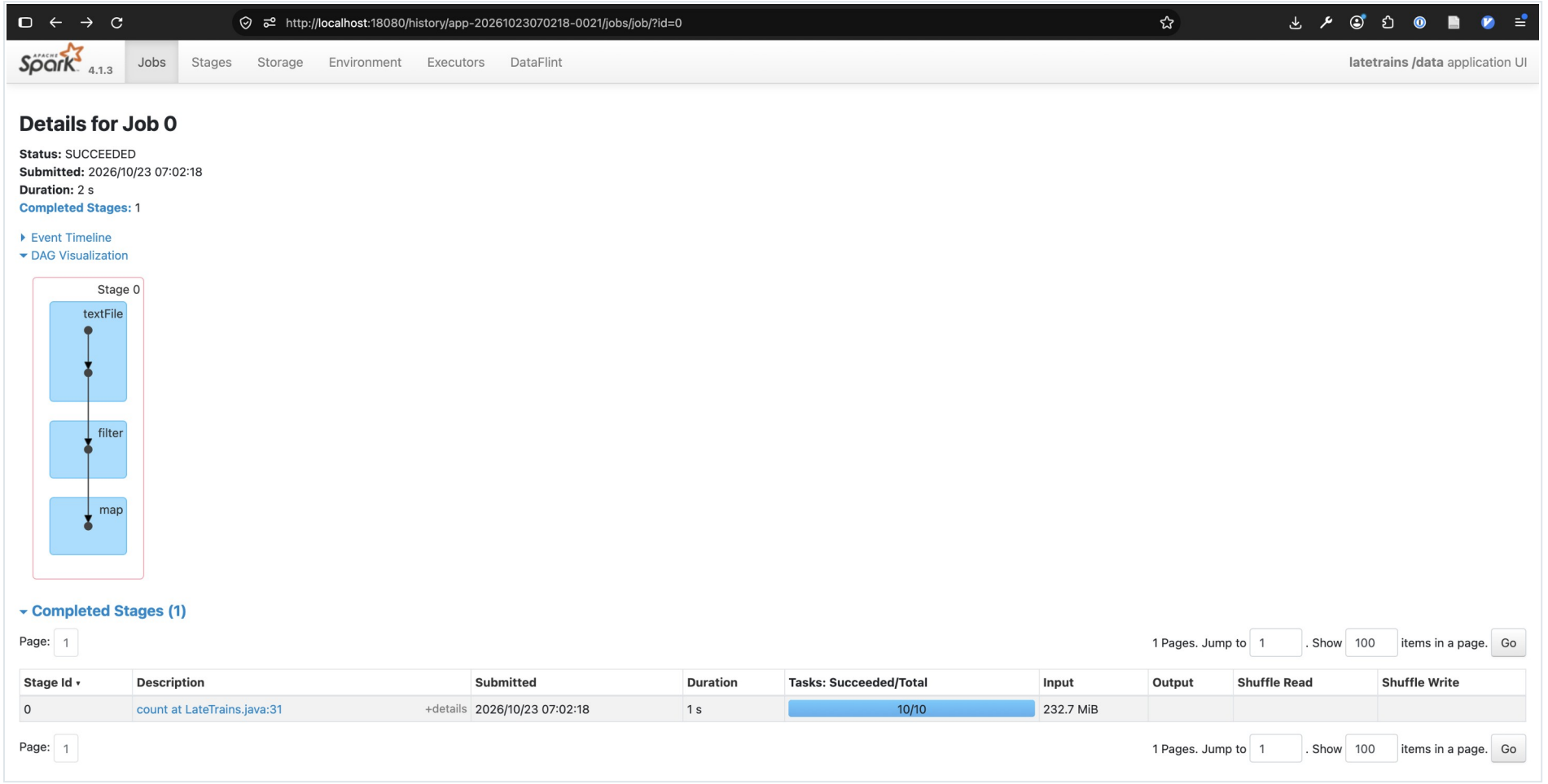
```
JavaRDD<String> lines = sc.textFile(base + "/trains");
JavaPairRDD<Integer, Integer> late = lines
    .filter(line -> Trains.delay(line) > 0)
    .mapToPair(line -> new Tuple2<>(Trains.station(line), Trains.delay(line)));
System.out.println(late.toDebugString());
System.out.println("late departures: " + late.count());
```

Prediction

- How many stages will run?
- How often will each line be read from disk?
- Which RDDs will appear in the lineage printed by `toDebugString`?

Universität St.Gallen

Let's see



Universität St.Gallen

LateTrains

```
JavaRDD<String> lines = sc.textFile(base + "/trains");
JavaPairRDD<Integer, Integer> late = lines
    .filter(line -> Trains.delay(line) > 0)
    .mapToPair(line -> new Tuple2<>(Trains.station(line), Trains.delay(line)));
System.out.println(late.toDebugString());
System.out.println("late departures: " + late.count());
```

Prediction

- How many stages will run?
- How often will each line be read from disk?
- Which RDDs will appear in the lineage printed by `toDebugString`?

Observed result

jobs	1: count
stages	1
tasks	10
result	3,121,362 late

In `toDebugString`, every narrow operation shares the indentation of the source RDD because no shuffle separates them. Each task filters and maps the records during its single split read.

Universität St.Gallen

From last week

Transformations on JavaPairRDD

operation	example	result
`reduceByKey(f)`	`pairs.reduceByKey((x, y) -> x + y)`	("a", 4), ("b", 2): `f` combines 2 values of a key

Universität St.Gallen

DelayMinutes

Sum the delay minutes for each station.



```
JavaPairRDD<Integer, Integer> late = sc.textFile(base + "/trains")
    .filter(line -> Trains.delay(line) > 0)
    .mapToPair(line -> new Tuple2<>(Trains.station(line), Trains.delay(line)));

// TODO start: add up the minutes of delay per station
JavaPairRDD<Integer, Integer> minutes = null;
// TODO end

// The indented lines are the stage before the shuffle.
System.out.println(minutes.toDebugString());
// Job 0. TreeMap sorts the stations by id on the driver.
Map<Integer, Integer> perStation = new TreeMap<>(minutes.collectAsMap());
perStation.forEach((station, total) -> System.out.printf("station %2d  %7d min%n", station, total));

// TODO start: the same sum, into 2 partitions
JavaPairRDD<Integer, Integer> minutesIn2 = null;
// TODO end

// Job 1: its second stage runs a task per partition of minutesIn2.
System.out.println("partitions: " + minutes.getNumPartitions() + " and " + minutesIn2.getNumPartitions());
System.out.println("stations: " + minutesIn2.count());
```

Universität St.Gallen

DelayMinutes

Sum the delay minutes for each station.



```
JavaPairRDD<Integer, Integer> late = sc.textFile(base + "/trains")
    .filter(line -> Trains.delay(line) > 0)
    .mapToPair(line -> new Tuple2<>(Trains.station(line), Trains.delay(line)));

// TODO start: add up the minutes of delay per station
JavaPairRDD<Integer, Integer> minutes = late.reduceByKey(Integer::sum);
// TODO end

// The indented lines are the stage before the shuffle.
System.out.println(minutes.toDebugString());
// Job 0. TreeMap sorts the stations by id on the driver.
Map<Integer, Integer> perStation = new TreeMap<>(minutes.collectAsMap());
perStation.forEach((station, total) -> System.out.printf("station %2d  %7d min%n", station, total));

// TODO start: the same sum, into 2 partitions
JavaPairRDD<Integer, Integer> minutesIn2 = late.reduceByKey(Integer::sum, 2);
// TODO end

// Job 1: its second stage runs a task per partition of minutesIn2.
System.out.println("partitions: " + minutes.getNumPartitions() + " and " + minutesIn2.getNumPartitions());
System.out.println("stations: " + minutesIn2.count());
```

Universität St.Gallen

DelayMinutes

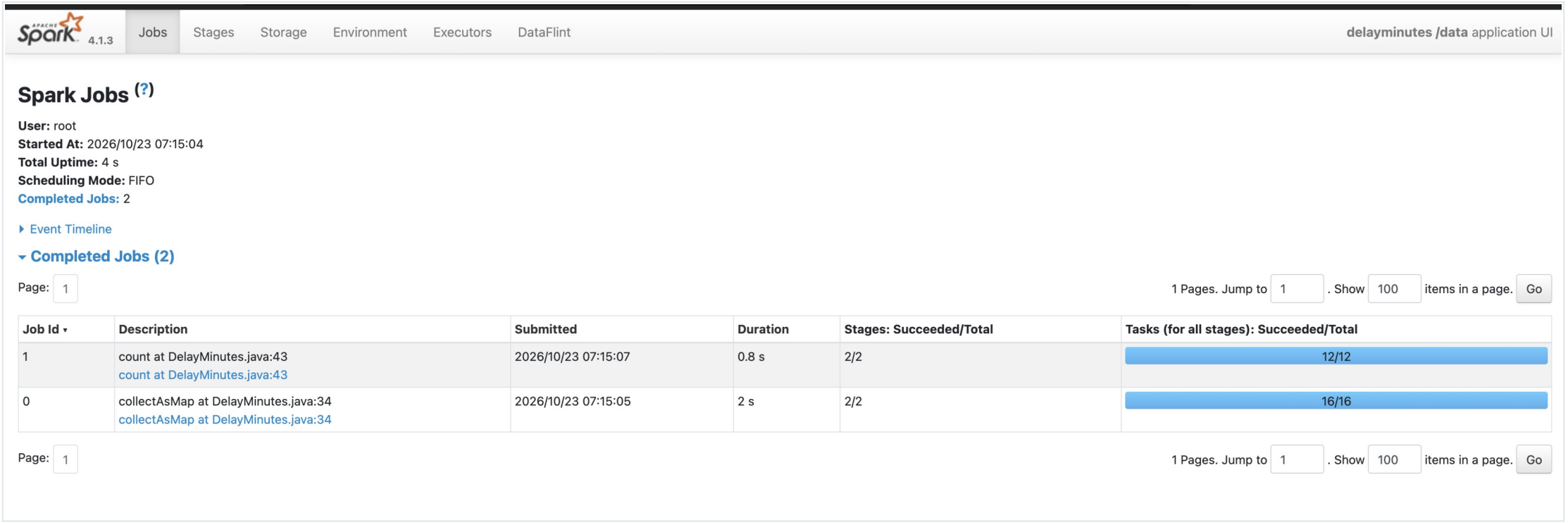
```
JavaPairRDD<Integer, Integer> late = sc.textFile(base + "/trains")
    .filter(line -> Trains.delay(line) > 0)
    .mapToPair(line -> new Tuple2<>(Trains.station(line), Trains.delay(line)));
JavaPairRDD<Integer, Integer> minutes = late.reduceByKey(Integer::sum);
Map<Integer, Integer> perStation = new TreeMap<>(minutes.collectAsMap());
JavaPairRDD<Integer, Integer> minutesIn2 = late.reduceByKey(Integer::sum, 2);
System.out.println("stations: " + minutesIn2.count());
```

Prediction

- How many stages and tasks will each job have?
- How many records will cross the shuffle for 40 station keys?

Universität St.Gallen

Let's see



Universität St.Gallen

DelayMinutes

```
JavaPairRDD<Integer, Integer> late = sc.textFile(base + "/trains")
    .filter(line -> Trains.delay(line) > 0)
    .mapToPair(line -> new Tuple2<>(Trains.station(line), Trains.delay(line)));
JavaPairRDD<Integer, Integer> minutes = late.reduceByKey(Integer::sum);
Map<Integer, Integer> perStation = new TreeMap<>(minutes.collectAsMap());
JavaPairRDD<Integer, Integer> minutesIn2 = late.reduceByKey(Integer::sum, 2);
System.out.println("stations: " + minutesIn2.count());
```

Prediction

- How many stages and tasks will each job have?
- How many records will cross the shuffle for 40 station keys?

Observed result

job 0	stage 0: 10 tasks, stage 1: 6 tasks
job 1	stage 2: 10 tasks, stage 3: 2 tasks
Shuffle Write	400 records per job

The map stage runs a task per file split. Without an explicit partition count, the stage after the shuffle uses the six tasks set by `spark.default.parallelism`. The second `reduceByKey` call sets two tasks. Local combination leaves 40 records per map task and 400 shuffle records across the ten map tasks.

Break

5 minutes

Universität St.Gallen

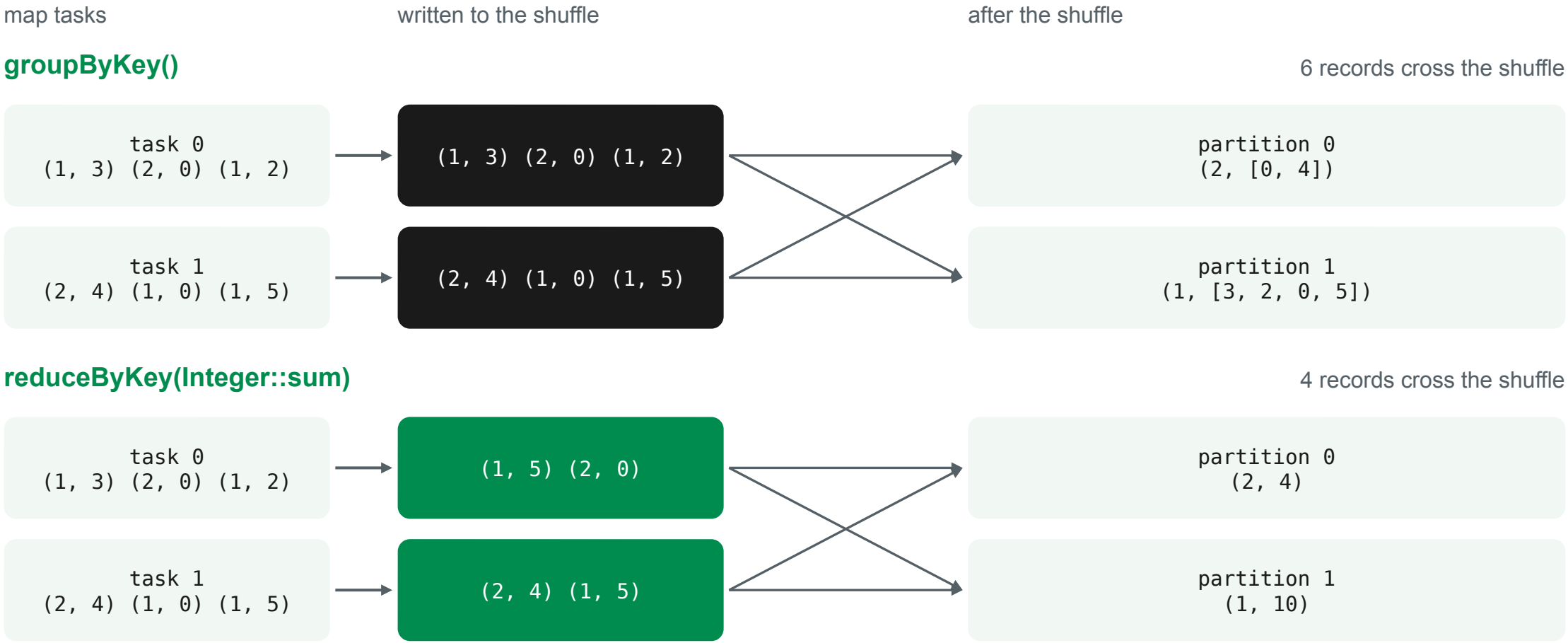
From last week

Transformations on JavaPairRDD

operation	example	result
`mapValues(f)`	`pairs.mapValues(v -> v * 10)`	("a", 10), ("b", 20), ("a", 30): new values, same keys

Universität St.Gallen

A bit of Theory



Universität St.Gallen

SumThreeWays

Compare delay totals from `groupByKey`, filtered `groupByKey`, and `reduceByKey`.



```
JavaPairRDD<Integer, Integer> departures = sc.textFile(base + "/trains")
    .mapToPair(line -> new Tuple2<>(Trains.station(line), Trains.delay(line)));

// Job 0: groupByKey sends every departure through the shuffle.
JavaPairRDD<Integer, Integer> grouped = departures.groupByKey().mapValues(SumThreeWays::sum);
System.out.println("groupByKey:              " + grouped.collectAsMap().get(1) + " min at station 1");

// TODO start: job 1, drop the departures without delay before groupByKey
JavaPairRDD<Integer, Integer> filtered = null;
// TODO end
System.out.println("filter, then groupByKey: " + filtered.collectAsMap().get(1) + " min at station 1");

// TODO start: job 2, add up with reduceByKey
JavaPairRDD<Integer, Integer> reduced = null;
// TODO end
System.out.println("reduceByKey:             " + reduced.collectAsMap().get(1) + " min at stat
```

SumThreeWays.java

```
static int sum(Iterable<Integer> values) {
    int sum = 0;
    for (int v : values) {
        sum += v;
    }
    return sum;
}
```

Universität St.Gallen

SumThreeWays

Compare delay totals from `groupByKey`, filtered `groupByKey`, and `reduceByKey`.



```
JavaPairRDD<Integer, Integer> departures = sc.textFile(base + "/trains")
    .mapToPair(line -> new Tuple2<>(Trains.station(line), Trains.delay(line)));

// Job 0: groupByKey sends every departure through the shuffle.
JavaPairRDD<Integer, Integer> grouped = departures.groupByKey().mapValues(SumThreeWays::sum);
System.out.println("groupByKey:              " + grouped.collectAsMap().get(1) + " min at station 1");

// TODO start: job 1, drop the departures without delay before groupByKey
JavaPairRDD<Integer, Integer> filtered = departures
    .filter(t -> t._2() > 0)
    .groupByKey()
    .mapValues(SumThreeWays::sum);
// TODO end
System.out.println("filter, then groupByKey: " + filtered.collectAsMap().get(1) + " min at station 1");

// TODO start: job 2, add up with reduceByKey
JavaPairRDD<Integer, Integer> reduced = departures.reduceByKey(Integer::sum);
// TODO end
System.out.println("reduceByKey:             " + reduced.collectAsMap().get(1) + " min at station 1");
```

Universität St.Gallen

SumThreeWays

```
JavaPairRDD<Integer, Integer> departures = sc.textFile(base + "/trains")
    .mapToPair(line -> new Tuple2<>(Trains.station(line), Trains.delay(line)));
JavaPairRDD<Integer, Integer> grouped = departures.groupByKey().mapValues(SumThreeWays::sum);
JavaPairRDD<Integer, Integer> filtered = departures
    .filter(t -> t._2() > 0)
    .groupByKey()
    .mapValues(SumThreeWays::sum);
JavaPairRDD<Integer, Integer> reduced = departures.reduceByKey(Integer::sum);
```

Prediction

- How do the jobs rank by Shuffle Write?
- Will every job return the same total for station 1?

Universität St.Gallen

Let's see

▾ Completed Stages (2)

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id ▾	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
5	collectAsMap at SumThreeWays.java:38 +details	2026/10/23 07:43:54	26 ms	6/6			11.2 KiB	
4	mapToPair at SumThreeWays.java:21 +details	2026/10/23 07:43:53	0.9 s	10/10	232.7 MiB			11.2 KiB

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

▾ Completed Stages (2)

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id ▾	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
3	collectAsMap at SumThreeWays.java:33 +details	2026/10/23 07:43:53	0.2 s	6/6			15.4 MiB	
2	filter at SumThreeWays.java:29 +details	2026/10/23 07:43:52	0.8 s	10/10	232.7 MiB			15.4 MiB

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

▾ Completed Stages (2)

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id ▾	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
1	collectAsMap at SumThreeWays.java:25 +details	2026/10/23 07:43:52	0.6 s	6/6			42.0 MiB	
0	mapToPair at SumThreeWays.java:21 +details	2026/10/23 07:43:50	1 s	10/10	232.7 MiB			42.0 MiB

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

Universität St.Gallen

SumThreeWays

```
JavaPairRDD<Integer, Integer> departures = sc.textFile(base + "/trains")
    .mapToPair(line -> new Tuple2<>(Trains.station(line), Trains.delay(line)));
JavaPairRDD<Integer, Integer> grouped = departures.groupByKey().mapValues(SumThreeWays::sum);
JavaPairRDD<Integer, Integer> filtered = departures
    .filter(t -> t._2() > 0)
    .groupByKey()
    .mapValues(SumThreeWays::sum);
JavaPairRDD<Integer, Integer> reduced = departures.reduceByKey(Integer::sum);
```

Prediction

- How do the jobs rank by Shuffle Write?
- Will every job return the same total for station 1?

Observed result

job 0	9,377,000 records, 42.0 MiB
job 1	3,121,362 records, 15.4 MiB
job 2	400 records, 11.2 KiB

Each job reports 419,479 minutes for station 1. Removing zero-delay departures before the aggregation leaves the sum unchanged.

Universität St.Gallen

From last week

Transformations on JavaPairRDD

operation	example	result
`join(other)`	`pairs.join(fruits)`	("a", (1, "apple")), ("a", (3, "apple")), ("b", (2, "banana"))

Universität St.Gallen

StationNames

Join each station's delay total with its station name.



```
JavaPairRDD<Integer, Integer> minutes = sc.textFile(base + "/trains")
    .mapToPair(line -> new Tuple2<>(Trains.station(line), Trains.delay(line)))
    .reduceByKey(Integer::sum);

// TODO start: read stations.csv (station_id,name) as station -> name
JavaPairRDD<Integer, String> stations = null;
// TODO end

// TODO start: join them into station -> (minutes, name)
JavaPairRDD<Integer, Tuple2<Integer, String>> named = null;
// TODO end

System.out.println(named.toDebugString());
List<Tuple2<Integer, Tuple2<Integer, String>>> rows = named.collect();
for (Tuple2<Integer, Tuple2<Integer, String>> row : rows) {
    System.out.printf("%2d  %-20s %7d min%n", row._1(), row._2()._2(), row._2()._1());
}
System.out.println(rows.size() + " stations");
```

Universität St.Gallen

StationNames

Join each station's delay total with its station name.



```
JavaPairRDD<Integer, Integer> minutes = sc.textFile(base + "/trains")
    .mapToPair(line -> new Tuple2<>(Trains.station(line), Trains.delay(line)))
    .reduceByKey(Integer::sum);

// TODO start: read stations.csv (station_id,name) as station -> name
JavaPairRDD<Integer, String> stations = sc.textFile(base + "/stations.csv")
    .mapToPair(line -> new Tuple2<>(Stations.id(line), Stations.name(line)));
// TODO end

// TODO start: join them into station -> (minutes, name)
JavaPairRDD<Integer, Tuple2<Integer, String>> named = minutes.join(stations);
// TODO end

System.out.println(named.toDebugString());
List<Tuple2<Integer, Tuple2<Integer, String>>> rows = named.collect();
for (Tuple2<Integer, Tuple2<Integer, String>> row : rows) {
    System.out.printf("%2d  %-20s %7d min%n", row._1(), row._2()._2(), row._2()._1());
}
System.out.println(rows.size() + " stations");
```

Universität St.Gallen

StationNames

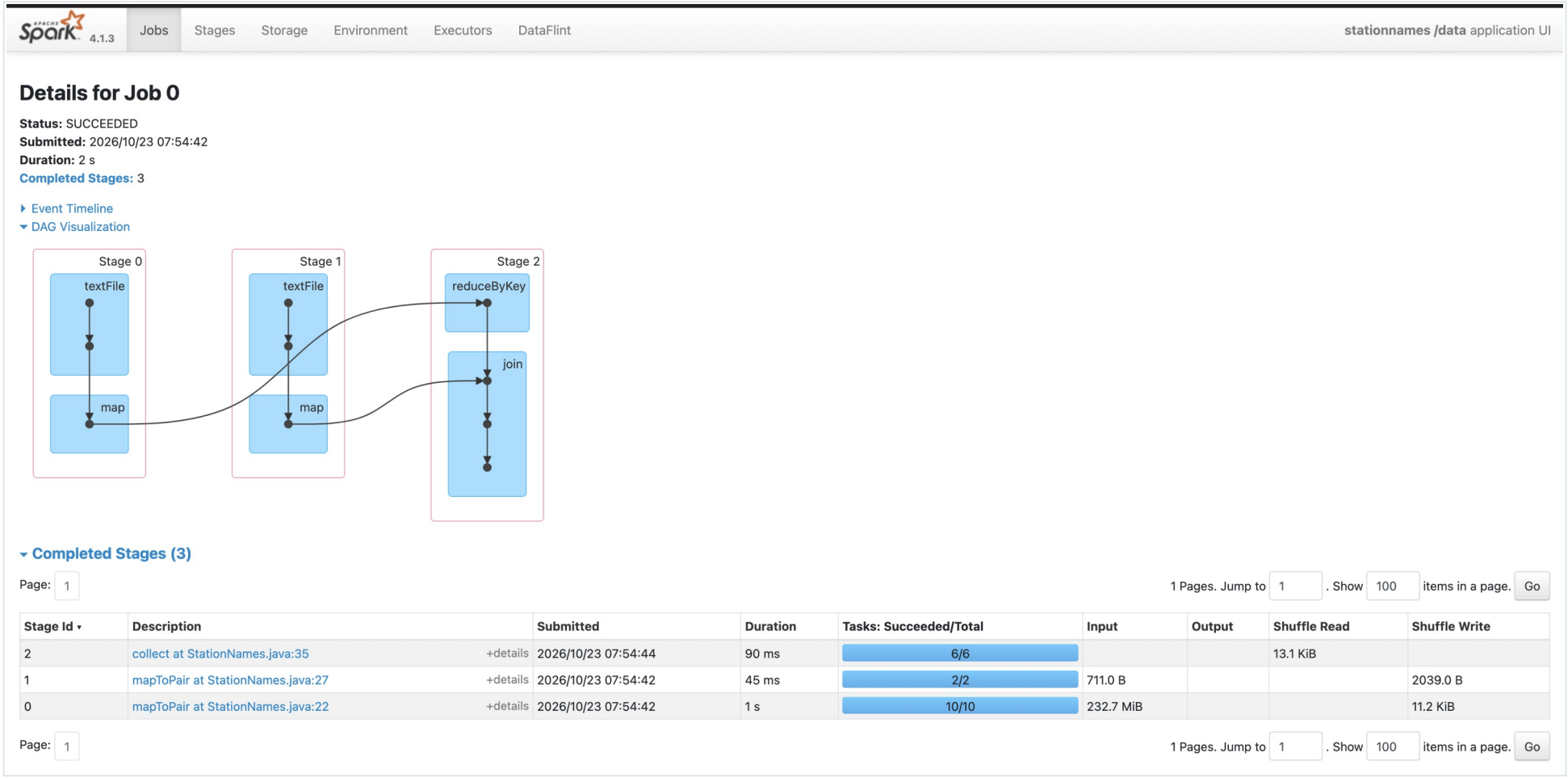
```
JavaPairRDD<Integer, Integer> minutes = sc.textFile(base + "/trains")
    .mapToPair(line -> new Tuple2<>(Trains.station(line), Trains.delay(line)))
    .reduceByKey(Integer::sum);
JavaPairRDD<Integer, String> stations = sc.textFile(base + "/stations.csv")
    .mapToPair(line -> new Tuple2<>(Stations.id(line), Stations.name(line)));
JavaPairRDD<Integer, Tuple2<Integer, String>> named = minutes.join(stations);
List<Tuple2<Integer, Tuple2<Integer, String>>> rows = named.collect();
```

Prediction

- How many stages will the job run? Does the join add a stage of its own?
- The departures have the station IDs 1 to 40, and `stations.csv` lists the IDs 1 to 38 and 41. How many stations will the inner join return?

Universität St.Gallen

Let's see



Universität St.Gallen

StationNames

```
JavaPairRDD<Integer, Integer> minutes = sc.textFile(base + "/trains")
    .mapToPair(line -> new Tuple2<>(Trains.station(line), Trains.delay(line)))
    .reduceByKey(Integer::sum);
JavaPairRDD<Integer, String> stations = sc.textFile(base + "/stations.csv")
    .mapToPair(line -> new Tuple2<>(Stations.id(line), Stations.name(line)));
JavaPairRDD<Integer, Tuple2<Integer, String>> named = minutes.join(stations);
List<Tuple2<Integer, Tuple2<Integer, String>>> rows = named.collect();
```

Prediction

- How many stages will the job run? Does the join add a stage of its own?
- The departures have the station IDs 1 to 40, and `stations.csv` lists the IDs 1 to 38 and 41. How many stations will the inner join return?

Observed result

stages	3, the join adds none
tasks	2 (stations.csv), 10 (trains), 6 (join)
result	38 stations: IDs 1 to 38

From last week

Transformations on JavaPairRDD

operation	example	result
`sortByKey(asc)`	`pairs.sortByKey(false)`	("b", 2), ("a", 1), ("a", 3): `true` for ascending

Actions in the Java API

operation	example	result
`top(n, cmp)`	`pairs.top(1, BY_VALUE)`	[("a", 3)]: the largest by `cmp`, a `Serializable` comparator

Universität St.Gallen

WorstStations

Return the ten stations with the highest total delay.

```
static final Comparator<Tuple2<Integer, Integer>> BY_MINUTES =
    (Comparator<Tuple2<Integer, Integer>> & Serializable) (a, b) -> Integer.compare(a._2(), b._2());

JavaPairRDD<Integer, Integer> minutes = sc.textFile(base + "/trains")
    .mapToPair(line -> new Tuple2<>(Trains.station(line), Trains.delay(line)))
    .reduceByKey(Integer::sum);

// TODO start: make the minutes the key, sort them in descending order and collect
List<Tuple2<Integer, Integer>> sorted = null;
// TODO end
System.out.println("sortByKey: " + sorted.subList(0, 10));

// TODO start: the 10 pairs with the most minutes, using BY_MINUTES
List<Tuple2<Integer, Integer>> top = null;
// TODO end
System.out.println("top:       " + top);
```

Universität St.Gallen

WorstStations

Return the ten stations with the highest total delay.

```
static final Comparator<Tuple2<Integer, Integer>> BY_MINUTES =
    (Comparator<Tuple2<Integer, Integer>> & Serializable) (a, b) -> Integer.compare(a._2(), b._2());

JavaPairRDD<Integer, Integer> minutes = sc.textFile(base + "/trains")
    .mapToPair(line -> new Tuple2<>(Trains.station(line), Trains.delay(line)))
    .reduceByKey(Integer::sum);

// TODO start: make the minutes the key, sort them in descending order and collect
List<Tuple2<Integer, Integer>> sorted = minutes
    .mapToPair(Tuple2::swap)
    .sortByKey(false)
    .collect();
// TODO end
System.out.println("sortByKey: " + sorted.subList(0, 10));

// TODO start: the 10 pairs with the most minutes, using BY_MINUTES
List<Tuple2<Integer, Integer>> top = minutes.top(10, BY_MINUTES);
// TODO end
System.out.println("top:       " + top);
```

Universität St.Gallen

WorstStations

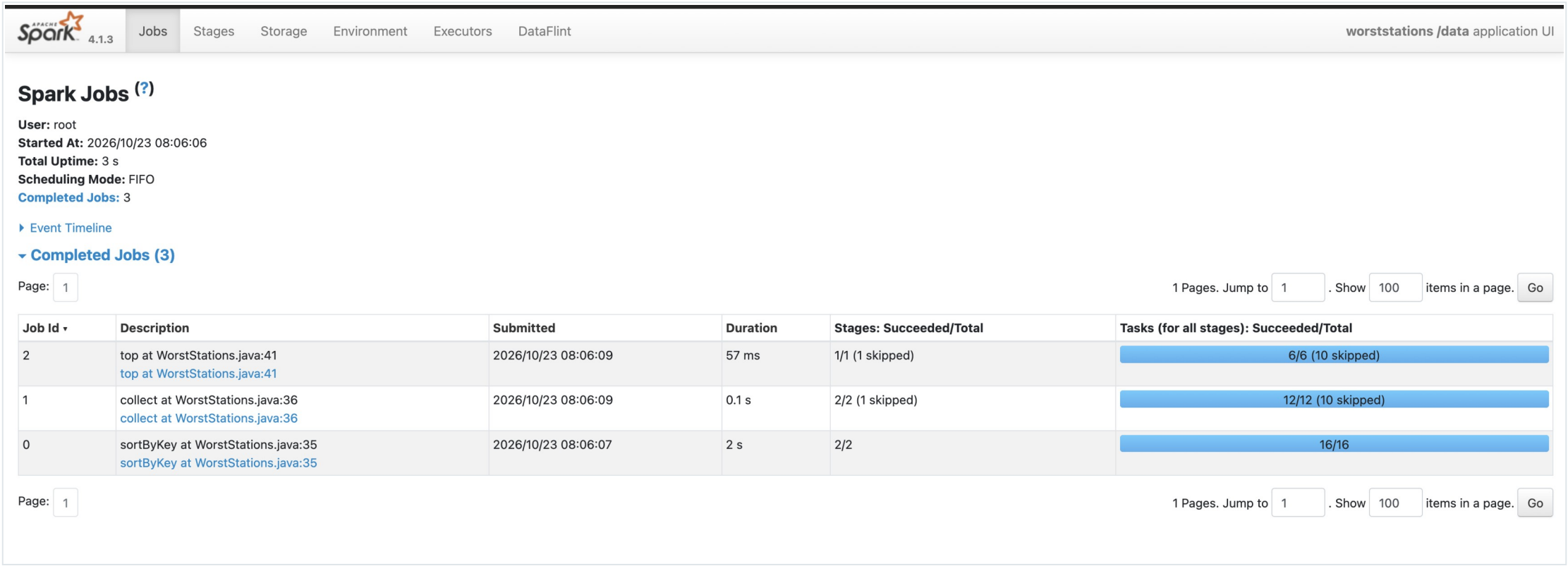
```
JavaPairRDD<Integer, Integer> minutes = sc.textFile(base + "/trains")
    .mapToPair(line -> new Tuple2<>(Trains.station(line), Trains.delay(line)))
    .reduceByKey(Integer::sum);
List<Tuple2<Integer, Integer>> sorted = minutes
    .mapToPair(Tuple2::swap)
    .sortByKey(false)
    .collect();
List<Tuple2<Integer, Integer>> top = minutes.top(10, BY_MINUTES);
```

Prediction

- How many jobs will run?
- Which stages can later jobs skip?
- Which version will shuffle less data: `sortByKey` or `top`?

Universität St.Gallen

Let's see



Universität St.Gallen

WorstStations

```
JavaPairRDD<Integer, Integer> minutes = sc.textFile(base + "/trains")
    .mapToPair(line -> new Tuple2<>(Trains.station(line), Trains.delay(line)))
    .reduceByKey(Integer::sum);
List<Tuple2<Integer, Integer>> sorted = minutes
    .mapToPair(Tuple2::swap)
    .sortByKey(false)
    .collect();
List<Tuple2<Integer, Integer>> top = minutes.top(10, BY_MINUTES);
```

Prediction

- How many jobs will run?
- Which stages can later jobs skip?
- Which version will shuffle less data: `sortByKey` or `top`?

Observed result

job 0	sortByKey samples the keys
job 1	collect: 3 stages, 1 skipped
job 2	top: 2 stages, 1 skipped

Universität St.Gallen

Punctuality

Count departures delayed by more than 1, 3, 5, and 10 minutes.



```
static final int[] LIMITS = {1, 3, 5, 10};

JavaRDD<Integer> delays = sc.textFile(base + "/trains").map(Trains::delay);

// Jobs 0 to 3: each count reads all files again.
for (int limit : LIMITS) {
    System.out.println("more than " + limit + " min late: " + delays.filter(d -> d > limit).count());
}

// TODO start: keep delays in the memory of the executors
// TODO end

// Jobs 4 to 7: job 4 reads the files and keeps the partitions, jobs 5 to 7 read them from memory.
for (int limit : LIMITS) {
    System.out.println("more than " + limit + " min late: " + delays.filter(d -> d > limit).count());
}

sc.stop();
```

Universität St.Gallen

Punctuality

Count departures delayed by more than 1, 3, 5, and 10 minutes.



```java
static final int[] LIMITS = {1, 3, 5, 10};

JavaRDD<Integer> delays = sc.textFile(base + "/trains").map(Trains::delay);

// Jobs 0 to 3: each count reads all files again.
for (int limit : LIMITS) {
    System.out.println("more than " + limit + " min late: " + delays.filter(d -> d > limit).count());
}

// TODO start: keep delays in the memory of the executors
delays.cache();
// TODO end

// Jobs 4 to 7: job 4 reads the files and keeps the partitions, jobs 5 to 7 read them from memory.
for (int limit : LIMITS) {
    System.out.println("more than " + limit + " min late: " + delays.filter(d -> d > limit).count());
}

sc.stop();
```

Universität St.Gallen

Punctuality

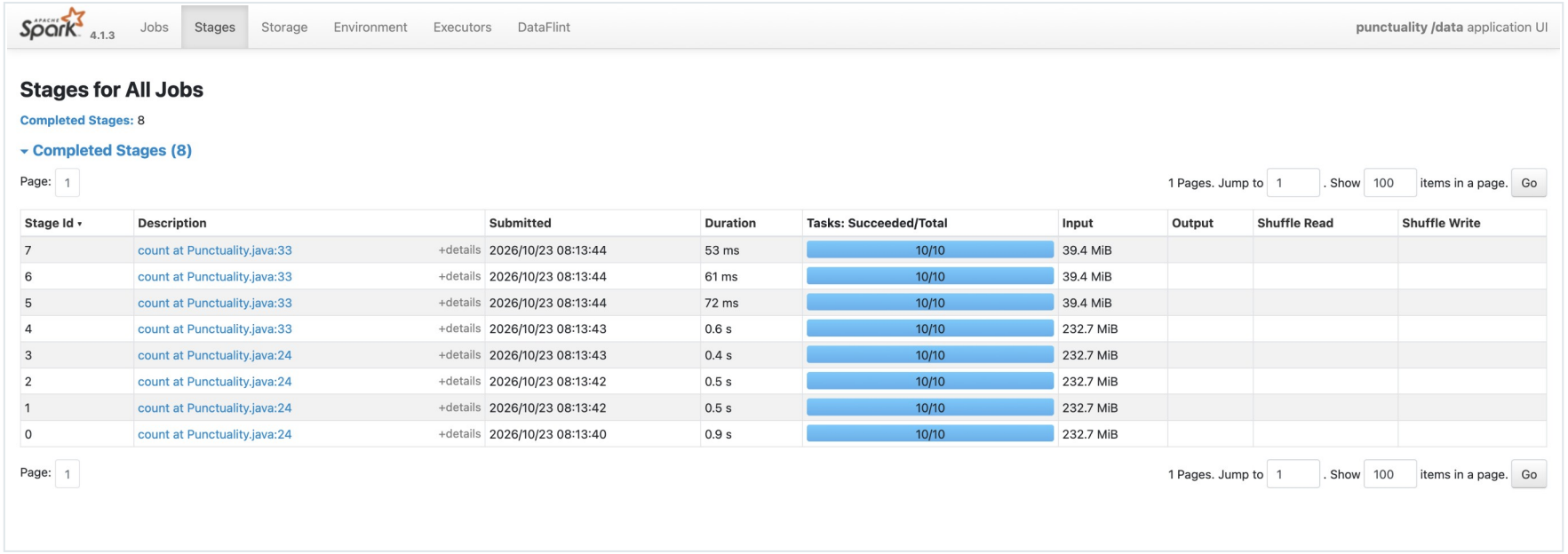
```
JavaRDD<Integer> delays = sc.textFile(base + "/trains").map(Trains::delay);
for (int limit : LIMITS) {
    System.out.println("more than " + limit + " min late: " + delays.filter(d -> d > limit).count());
}
delays.cache();
for (int limit : LIMITS) {
    System.out.println("more than " + limit + " min late: " + delays.filter(d -> d > limit).count());
}
```

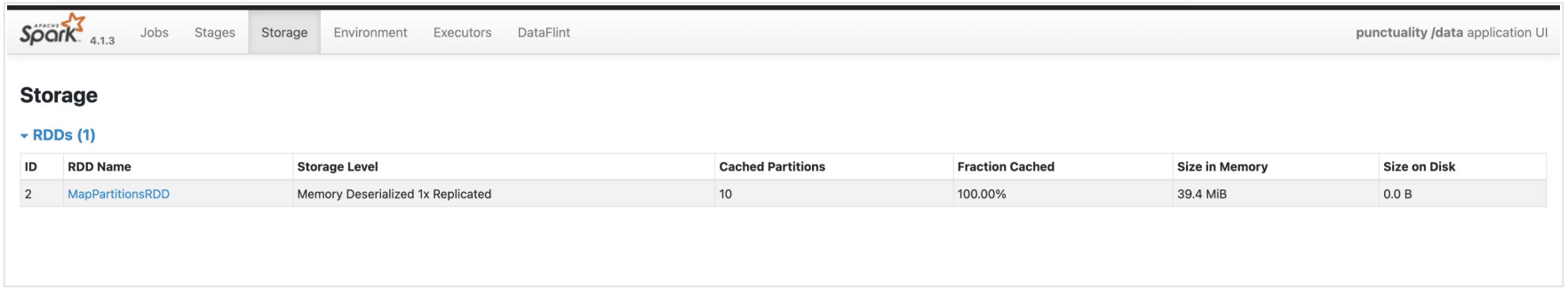
Prediction

- How many jobs will run?
- Which jobs will read the source files?
- What Input value will the remaining jobs show?

Universität St.Gallen

Let's see





Universität St.Gallen

Punctuality

```
JavaRDD<Integer> delays = sc.textFile(base + "/trains").map(Trains::delay);
for (int limit : LIMITS) {
    System.out.println("more than " + limit + " min late: " + delays.filter(d -> d > limit).count());
}
delays.cache();
for (int limit : LIMITS) {
    System.out.println("more than " + limit + " min late: " + delays.filter(d -> d > limit).count());
}
```

Prediction

- How many jobs will run?
- Which jobs will read the source files?
- What Input value will the remaining jobs show?

Observed result

jobs	8, a stage of 10 tasks each
jobs 0-4	Input 232.7 MiB from the files
jobs 5-7	Input 39.4 MiB from memory

Job 4 reads the files and stores the `delays` partitions on the executors.
Jobs 5-7 read those cached partitions and produce the same counts.

Universität St.Gallen

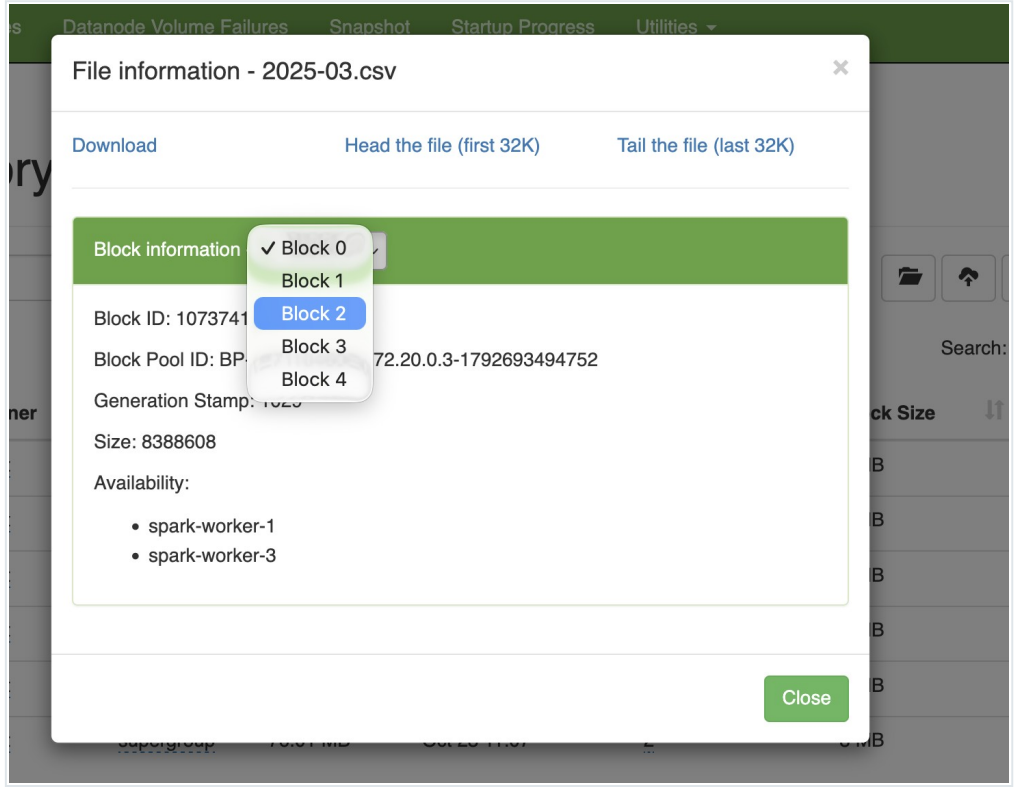
Appendix: HDFS and DataFlint

Universität St.Gallen

HDFS blocks and replicas

```
hdfs dfs -mkdir -p /data
hdfs dfs -put -f /data/trains /data/stations.csv /data/
```

- In the NameNode UI on port 9870, open **Utilities -> Browse the file system -> /data/trains**.
- With an 8 MiB block size and replication factor 2, locate every block and replica of `2025-02.csv` (11.9 MiB).



Universität St.Gallen

Departures from HDFS

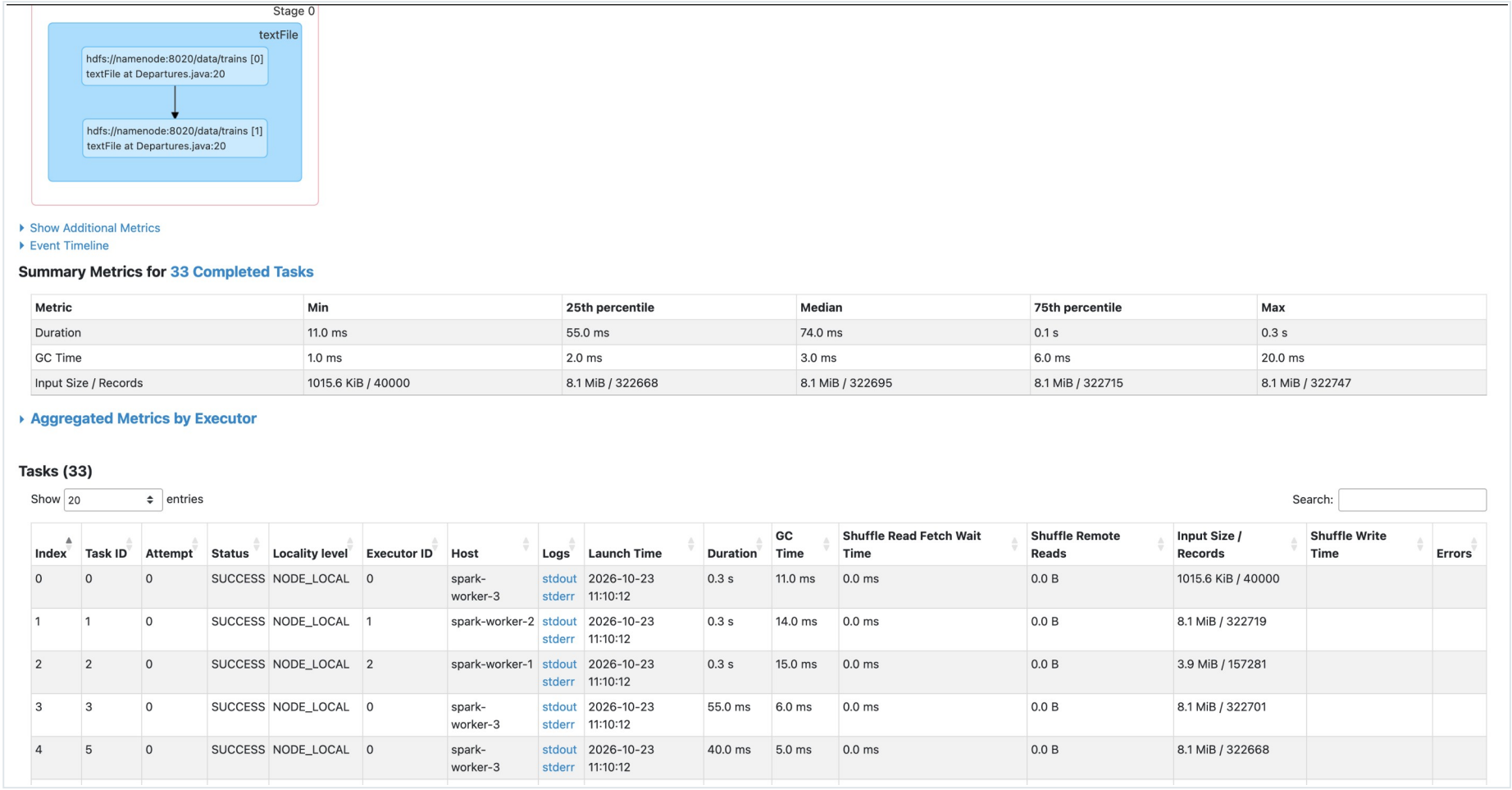
```
./run.sh solution departures hdfs
./run.sh solution departures
```

Prediction

- How many partitions will the HDFS and local `/data` runs create?
- Which **Locality Level** will each run show?

Universität St.Gallen

Let's see



▸ Show Additional Metrics
▸ Event Timeline

Summary Metrics for 33 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	11.0 ms	55.0 ms	74.0 ms	0.1 s	0.3 s
GC Time	1.0 ms	2.0 ms	3.0 ms	6.0 ms	20.0 ms
Input Size / Records	1015.6 KiB / 40000	8.1 MiB / 322668	8.1 MiB / 322695	8.1 MiB / 322715	8.1 MiB / 322747

▸ Aggregated Metrics by Executor

Tasks (33)

Show 20 entries Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Read Fetch Wait Time	Shuffle Remote Reads	Input Size / Records	Shuffle Write Time	Errors
0	0	0	SUCCESS	NODE_LOCAL	0	spark-worker-3	stdout stderr	2026-10-23 11:10:12	0.3 s	11.0 ms	0.0 ms	0.0 B	1015.6 KiB / 40000		
1	1	0	SUCCESS	NODE_LOCAL	1	spark-worker-2	stdout stderr	2026-10-23 11:10:12	0.3 s	14.0 ms	0.0 ms	0.0 B	8.1 MiB / 322719		
2	2	0	SUCCESS	NODE_LOCAL	2	spark-worker-1	stdout stderr	2026-10-23 11:10:12	0.3 s	15.0 ms	0.0 ms	0.0 B	3.9 MiB / 157281		
3	3	0	SUCCESS	NODE_LOCAL	0	spark-worker-3	stdout stderr	2026-10-23 11:10:12	55.0 ms	6.0 ms	0.0 ms	0.0 B	8.1 MiB / 322701		
4	5	0	SUCCESS	NODE_LOCAL	0	spark-worker-3	stdout stderr	2026-10-23 11:10:12	40.0 ms	5.0 ms	0.0 ms	0.0 B	8.1 MiB / 322668		

Universität St.Gallen

Departures from HDFS

```
./run.sh solution departures hdfs
./run.sh solution departures
```

Prediction

- How many partitions will the HDFS and local `/data` runs create?
- Which **Locality Level** will each run show?

Observed result

HDFS	33 partitions, NODE_LOCAL
/data	10 partitions, PROCESS_LOCAL

From HDFS, the 8 MiB block size caps a split: `2025-02.csv` (11.9 MiB) becomes an 8 MiB and a 3.9 MiB split, and the six files give 1 + 2 + 5 + 6 + 9 + 10 = 33 partitions. Spark asks the NameNode for block locations and assigns each task to a worker with a replica. Local files under `/data` provide no distributed block location.

Universität St.Gallen

DelayReport

Report the delays per station and per hour, and the worst streak of 10 departures, in Spark SQL.



```
spark.read().schema("date DATE, time STRING, line STRING, station_id INT, delay_min INT")
    .csv(base + "/trains").createOrReplaceTempView("trains");
spark.read().schema("station_id INT, name STRING")
    .csv(base + "/stations.csv").createOrReplaceTempView("stations");

// Query 3: per station, the worst streak: the highest average delay over 10 departures in a row.
// The window needs all departures of a station in time order.
spark.sql("""
    SELECT station_id, ROUND(MAX(streak), 1) AS worst_streak
    FROM (SELECT station_id,
                 AVG(delay_min) OVER (PARTITION BY station_id ORDER BY date, time
                                      ROWS BETWEEN 9 PRECEDING AND CURRENT ROW) AS streak
          FROM trains)
    GROUP BY station_id
    ORDER BY worst_streak DESC
    """).show(10);
```

Universität St.Gallen

Inspect a run with DataFlint

Run `./run.sh solution delayreport`, then open the application in the History Server and select its **DataFlint** tab.

Tasks

- Follow each query from the scan of `trains` to its result. Which step shuffles data?
- How many tasks does the scan of `trains` run? Compare them with the 10 partitions of `textFile`.
- Query 3 shuffles every departure. Compare its Shuffle Write with queries 1 and 2, and look for spill.
- Follow each DataFlint alert to the query it refers to.

